

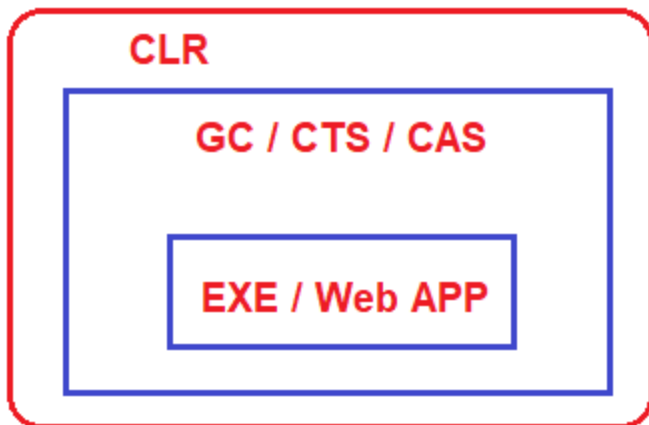
Managed and Unmanaged Code in .NET Framework

In this article, I am going to discuss the **Managed and Unmanaged Code in .NET Framework**. Please read our previous article where we discussed **Common Language Specification (CLS)** in detail. Managed Code means the code which is designed and developed under the .NET Framework. Unmanaged Code means code that is not written and managed using the .NET Framework. At the end of this article, you will understand what Managed Code and Unmanaged code are and how they are executed in .NET Framework with Examples.

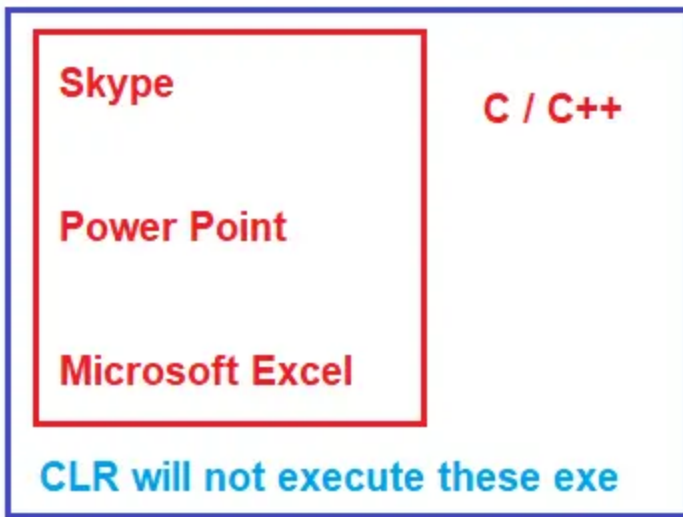
Understanding the Managed and Unmanaged Code in .NET Framework:

Whenever we create any EXE (i.e. Console Application, Windows Application, Class Library Project, etc.) or any DLL i.e. Web Application (i.e. ASP.NET MVC, ASP.NET Web API, ASP.NET, etc.) in .NET Framework using Visual Studio and using any .NET supported Programming Language such as C#, VB, F#, J#, etc., then these applications are run completely under the control of CLR (Common Language Runtime). CLR is the Runtime Engine of .NET Applications.

If your applications have unused objects, then CLR will clean those objects using **Garbage Collector**. If your application wants to communicate with other applications, then it will make sure that CTS (**Common Type System**) and CLS (**Common Language Specifications**) are followed. CLR also uses **CAS (Code Access Security)** and **CV (Code Verification)**, to manage the security of your application i.e. check whether the current user has rights to access the assembly or not and whether it is safe to execute the assembly by the Operating System or not. The CLR will also load your application and unload your application, etc. So, for better understanding, please have a look at the following image.



Now, let's say, you have also used other third-party EXE in your .NET application like Skype, PowerPoint, Microsoft Excel, Whatsapp, etc. These "EXEs" are not made in dot net, they are made using other programming languages let us as C and C++.

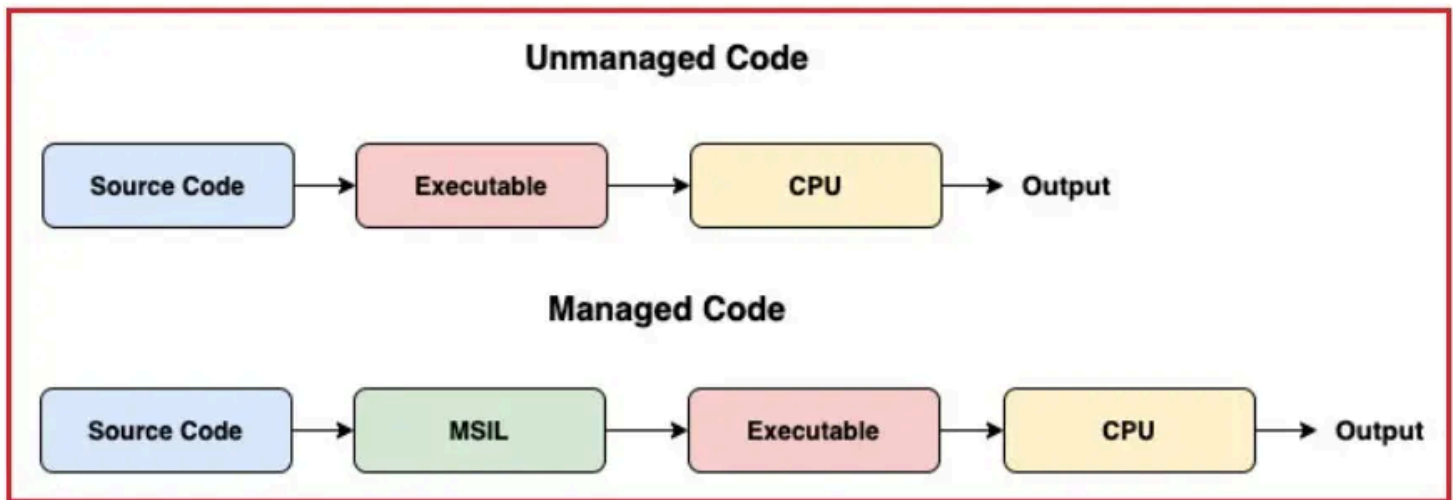


When you use these “EXEs” in your application, then these are not going to be run by CLR. Even though you are running these “EXEs” in .NET Applications, they are going to run under their own environment. For example, if one EXE is developed using C or C++, then that EXE will run under the C or C++ Runtime Environment, not the .NET Runtime Environment. In the same line, if the EXE is created using VB6, then it is going to run under the VB6 Runtime Environment, not using CLR which is the Runtime Environment for .NET Applications.

What exactly is the Managed and Unmanaged code in .NET Framework?

The codes which run under the complete control of CLR are called Managed Code in .NET Framework. These kinds of code (Managed code) are run by the .NET Runtime Environment i.e. CLR. If the .NET Framework is not installed or if the .NET Runtime Environment i.e. CLR is not available, then these kinds of codes are not going to be executed. CLR will provide all the facilities and features of .NET to the managed code execution like Language Interoperability, Automatic Memory Management, Exception Handling Mechanism, Code Access Security, Garbage Collection, etc. In this case, the source code is compiled in the intermediate language known as IL or MSIL, or CIL.

On the other hand, Skype, PowerPoint, and Microsoft Excel do not require .NET Runtime Environment, they run under their own environment. So, in short, the code (EXE, DLL) which does not run under the control of CLR is called Unmanaged Code. CLR will not provide any facilities and features of .NET to the Unmanaged Code like Language Interoperability, Automatic Memory Management, Exception Handling Mechanism, Code Access Security, Garbage Collection, etc. In this case, the source code directly compiles into native languages.



Note: Managed code is the code that is managed by the CLR (Common Language Runtime) in .NET Framework. Whereas Unmanaged code is the code that is directly executed by the operating system.

What are the advantages of using Managed Code?

1. It improves the security of the application like it will check whether the current has access to assembly or not and whether it is safe to execute the assembly by the Operating System or not.
2. Whenever an object is not used by the application, then the Garbage Collector automatically destroys the memory for unused objects.
3. It will support default exception handling.

What are the disadvantages of Managed Code?

1. The main disadvantage of managed code in the .NET Framework is that we are not allowed to allocate memory directly, or we cannot get low-level access to the CPU architecture. This is going to be managed by the CLR of the .NET Framework.

What are the advantages of using Unmanaged Code?

1. It provides the low-level access to the programmer.
2. It also provides direct access to the hardware of the machine.
3. The code is a little bit faster than managed code.
4. We can run the Unmanaged Code under any environment and platform.

What are the disadvantages of Unmanaged Code?

1. It does not provide security to the application.
2. Due to the access to memory allocation, issues related to memory leakage might have occurred.
3. No automatic garbage collection.
4. It will not support default exception handling, the developer needs to take care of that things.

In the next article, I am going to discuss [Assembly DLL and EXE in .NET Framework](#) with Examples. Here, in this article, I try to explain the **Managed and Unmanaged Code in C#.NET**. I hope you enjoy this article and I hope now you understood what exactly are Managed Code and Unmanaged Code in .NET Framework.